



華南師範大學

本科学学生实验（实践）报告

院 系：计 算 机 学 院

实验课程：编译原理

实验项目：LALR(1) 语法分析生成器

指导老师：黄煜廉

开课时间：2025 ~ 2026 年度第 2 学期

专 业：计算机科学与技术

班 级：24 级计算机科学与技术 2 班

学 生：廖艺琳

学 号：20230739041

华南师范大学教务处

华南师范大学实验报告

学生姓名_____学 号_____
专 业_____年级、班级_____
课程名称_____实验项目_____
实验类型 ☐验证 ☐设计 ☐综合 实验时间_____年____月____日
实验指导老师_____实验评分_____

一、实验内容

设计并实现一个 LALR(1)语法分析生成器应用程序，要求具备以下基本功能：

- (1) 提供文法编辑界面，支持用户输入和编辑文法，具备保存与打开文法文件的功能。
- (2) 计算文法非终结符的 FIRST 集和 FOLLOW 集，以表格形式展示。
- (3) 构建 LR(0) DFA (项目集规范族)，以表格形式展示各状态的项目和转移。
- (4) 判断文法是否为 SLR(1)文法，若非 SLR(1)则给出冲突原因。
- (5) 构建 LR(1) DFA (带展望符的项目集)，以表格形式展示。
- (6) 合并同核 LR(1)状态构建 LALR(1) DFA，检测合并后冲突。
- (7) 以 Windows 桌面应用程序方式运行，界面简洁清晰。

选做功能：

- (1) 生成 LALR(1)分析表 (ACTION 表和 GOTO 表)，以表格形式展示。
- (2) 允许用户输入待分析的句子，进行 LALR(1)语法分析。
- (3) 逐步显示分析过程，包括状态栈、输入串和分析动作。
- (4) 撰写完整的软件设计文档。

二、实验目的

1. 深入理解自底向上 LR 语法分析的理论原理，掌握 LR(0)、SLR(1)、LR(1)和 LALR(1)四种文法的层次关系与构造方法。
2. 掌握 FIRST 集和 FOLLOW 集的计算算法，理解这些集合在 LR 分析

表构建中的关键作用。

3. 掌握 LR 项目集规范族的构建方法，理解闭包 (Closure) 和转移 (Goto) 操作的核心逻辑。
4. 理解 LR(1) 项目中展望符 (Lookahead) 的传播机制，以及如何通过合并同核状态从 LR(1) 得到 LALR(1) 文法。
5. 掌握 LR 分析表的构建方法，能够根据分析表实现 LALR(1) 语法分析器，对输入句子进行移进-归约分析。
6. 提高使用 C++ 和 Qt 框架开发编译工具的实际工程能力。

三、实验文档：

3.1 系统总体结构

本系统采用模块化分层架构，分为数据模型层、算法逻辑层和用户界面层三个层次。各层职责分明，通过明确的数据接口进行交互，降低了模块间的耦合度。

文件组织结构：

```
LALR/
+-- Production.h          // 数据结构定义
+-- Grammar.h / .cpp      // 文法解析、FIRST/FOLLOW 计算
+-- LR DFA.h / .cpp       // LR(0)/LR(1)/LALR(1) DFA 构建与解析
+-- mainwindow.h / .cpp   // Qt 界面层
+-- mainwindow.ui         // UI 布局
+-- main.cpp              // 程序入口
+-- grammar_samples.txt    // 用例文件
```

系统数据流如下：

```
[文法文本]
|
v
Grammar::parse() -----> 产生式列表 + 终结符/非终结符
|
v
Grammar::computeFirst() -> FIRST 集
Grammar::computeFollow() -> FOLLOW 集
|
v
LRDFABuilder::buildAll()
+-- buildLR0States() -> LR(0) DFA
```

```

    +-- checkSLR1()      -> SLR(1) 冲突检测
    +-- buildLR1States() -> LR(1)  DFA
    +-- mergeToLALR()   -> LALR(1) DFA
    |
    v
    buildActionTable() + buildGotoTable() -> LALR(1) 分析表
    |
    v
    LRDFABuilder::parse() -> 句子分析步骤

```

用户界面层（**MainWindow**）负责接收用户输入，调用算法逻辑层完成计算，并将结果以文本和表格形式展示给用户。算法逻辑层（**Grammar** 和 **LRDFABuilder**）实现所有的编译理论算法，不依赖任何界面代码。数据模型层（**Production.h** 中的结构体）定义了整个系统中使用的核心数据类型。

3.2 数据结构设计

3.2.1 产生式结构（Production）

产生式是文法的基本单位，定义如下：

```

struct Production {
    QString lhs;           // 左部（非终结符）
    QStringList rhs;       // 右部符号序列（空列表表示 epsilon）
    int index = 0;         // 产生式编号
};

```

3.2.2 LR(1) 项目结构（LR1Item）

LR(1)项目同时用于 LR(0)和 LR(1) DFA 的构建。对于 LR(0)项目，lookahead 字段为空。

```

struct LR1Item {
    int prodIndex = 0;     // 产生式在增广文法中的下标
    int dotPos = 0;        // 圆点位置（0 表示在第一个符号之前）
    QString lookahead;     // 展望符（LR(1)使用，LR(0)忽略）
};

```

3.2.3 LR 状态结构（LRState）

```

struct LRState {

```

```

    int id = 0;

    QVector<LR1Item> items;           // 项目集合
    QMap<QString, int> transitions;    // 符号→目标状态
    QVector<int> originalIds;         // LALR 合并来源
};

```

3.2.4 分析表动作结构 (ParsingAction)

```

struct ParsingAction {
    QString type; // "shift" / "reduce" / "accept" / "conflict"
    int value = -1; // shift:目标状态; reduce:产生式下标
};

```

3.3 关键算法设计

3.3.1 文法解析与符号分类

系统首先将用户输入的文本按行分割，对每一行查找箭头符号（支持“->”和 Unicode 箭头“→”）以分离左部和右部。右部按“|”分割为多个候选式，每个候选式进一步分词得到符号序列。

符号分类规则：凡在产生式左部出现过的符号均为非终结符，其余符号（除 **epsilon** 外）均为终结符。第一个产生式的左部被设为文法的开始符号。

```

bool Grammar::parse(const QString &text) {
    // 按行分割、替换箭头、按|分割候选式
    line.replace("->", " ");
    line.replace(QChar(0x2192), ' '); // →
    QString lhs = line.left(arrowPos).trimmed();
    // ... 按|分割候选式，考虑括号深度 ...
    for (const auto &prod : productions)
        for (const auto &sym : prod.rhs)
            if (!nonTerminals.contains(sym))
                terminals.insert(sym);
}

```

3.3.2 FIRST 集与 FOLLOW 集计算

FIRST 集的计算采用迭代不动点算法。对每个非终结符，扫描其所有

产生式右部的第一个符号：若该符号为终结符则直接加入 FIRST 集；若为非终结符则递归地将其 FIRST 集（不含 ϵ ）加入。若某产生式右部所有符号均可推导出 ϵ ，则将 ϵ 也加入 FIRST 集。迭代直到所有 FIRST 集不再变化。

FOLLOW 集的计算同样采用迭代算法。初始化时将“\$”加入开始符号的 FOLLOW 集。对每条产生式 $A \rightarrow \alpha B \beta$ ，将 $\text{FIRST}(\beta)$ 除去 ϵ 后加入 B 的 FOLLOW 集；若 ϵ 属于 $\text{FIRST}(\beta)$ 或 β 为空，则将 A 的 FOLLOW 集全部加入 B 的 FOLLOW 集。迭代至不动点。

```
void Grammar::computeFirst() {
    for (const auto &prod : productions) {
        QSet<QString> &firstLHS = firstSets[prod.lhs];
        bool allEpsilon = true;
        for (const QString &sym : prod.rhs) {
            const QSet<QString> &firstSym = firstSets[sym];
            for (const QString &f : firstSym)
                if (f != "" && !firstLHS.contains(f))
                    firstLHS.insert(f), changed = true;
            if (!firstSym.contains(""))
                { allEpsilon = false; break; }
        }
        if (allEpsilon) firstLHS.insert(""), changed = true;
    }
}
```

3.3.3 LR(0) DFA 构建

首先构造增广文法 $S' \rightarrow S$ ，其中 S 为原文法的开始符号。从项目 $[S' \rightarrow \cdot S]$ 开始，通过闭包操作（Closure）和转移操作（Goto）逐步生成所有可达的项目集。闭包操作：若项目 $[A \rightarrow \alpha \cdot B \beta]$ 在状态中，则将所有 $B \rightarrow \cdot \gamma$ 加入同一状态。转移操作：从状态 I 经过符号 X 转移到状态 J，J 包含所有 $[A \rightarrow \alpha X \cdot \beta]$ （其中 $[A \rightarrow \alpha X \beta]$ 属于 I），并对 J 求闭包。重复此过程直到不再产生新状态。

```
QVector<LR1Item> LRDFABuilder::closureLR0(
    const QVector<LR1Item> &items,
    const QVector<Production> &augProds,
```

```

    const QSet<QString> &nonTerminals)
{
    QVector<LR1Item> result = items;
    QVector<LR1Item> queue = items;
    while (!queue.isEmpty()) {
        LR1Item item = queue.takeFirst();
        const Production &prod = augProds[item.prodIndex];
        if (item.dotPos < prod.rhs.size()) {
            const QString &next = prod.rhs[item.dotPos];
            if (nonTerminals.contains(next))
                for (int i = 0; i < augProds.size(); i++)
                    if (augProds[i].lhs == next)
                        result.append({i, 0});
        }
    }
    return result;
}

```

3.3.4 SLR(1) 冲突检测

对 LR(0) DFA 的每个状态，检查是否存在移进-归约冲突或归约-归约冲突。对于每个归约项目 $[A \rightarrow \alpha.]$ ，若其展望符（即 $\text{FOLLOW}(A)$ ）与状态中的移进符号存在交集，则报告移进-归约冲突。若同一状态中存在多个归约项目且其 FOLLOW 集有交集，则报告归约-归约冲突。

3.3.5 LR(1) DFA 构建

LR(1)的构建与 LR(0)类似，但项目携带展望符。初始项目为 $[S' \rightarrow .S, \$]$ 。闭包操作中，对项目 $[A \rightarrow \alpha.B\beta, a]$ ，计算 $\text{FIRST}(\beta a)$ 作为 B 的各个产生式的展望符，将 $[B \rightarrow .\gamma, b]$ 加入状态（b 属于 $\text{FIRST}(\beta a)$ ）。转移操作保持展望符不变。

```

QVector<LR1Item> LRDFABuilder::closureLR1(
    const QVector<LR1Item> &items,
    const QVector<Production> &augProds,
    const QSet<QString> &nonTerminals,
    const QMap<QString, QSet<QString>> &firstSets)

```

```

{
    while (!queue.isEmpty()) {
        LRItem item = queue.takeFirst();
        const Production &prod = augProds[item.prodIndex];
        if (item.dotPos < prod.rhs.size() ...) {
            QStringList beta = prod.rhs.mid(item.dotPos + 1);
            QStringList laSymbols = beta;
            laSymbols.append(item.lookahead);
            QSet<QString> lookaheads = firstOfString(
                laSymbols, firstSets);
            // 对 B 的每个产生式，添加[B→.γ, b]
            // (b ∈ FIRST(βa))
        }
    }
}

```

3.3.6 LALR(1) DFA 构建（同核合并）

LALR(1) DFA 通过合并 LR(1) DFA 中具有相同 LR(0)核心的状态得到。两个状态的 LR(0)核心相同，当且仅当它们包含的产生式-圆点位置对完全相同（忽略展望符差异）。合并时将对应状态的项目集取并集，展望符合并。然后重建状态间的转移关系。

合并后需进行冲突检测：若合并后的状态中，对同一个终结符既有移进动作又有归约动作，或存在多个不同的归约动作，则报告合并后冲突。

```

QVector<LRState> LRDFABuilder::mergeToLALR(
    const QVector<LRState> &lrlStates)
{
    // 1. 按 LR(0) 核心分组
    QMap<QString, QVector<int>> coreGroups;
    for (const auto &state : lrlStates) {
        auto core = lr0Core(state.items);
        coreGroups[key].append(state.id);
    }
    // 2. 合并每组为新的 LALR 状态
    for (auto [key, group] : coreGroups) {
        // 合并展望符
        QMap<QString, QSet<QString>> combinedLookaheads;
    }
}

```



```

        for (int sid : group)
            for (const auto &item : lrlStates[sid].items)
                combinedLookaheads[coreKey].insert(item.lookahead);
// 生成合并状态
lalrStates.append(newState);
}
// 3. 重建转移（遍历所有原始状态）
for (int i = 0; i < ns; i++)
    for (int oldId : originalIds)
        for (auto [sym, tgt] : lrlStates[oldId].transitions)
            if (oldToNew.contains(tgt))
                lalrStates[i].transitions[sym] = oldToNew[tgt];
}

```

3.3.7 LALR(1) 分析表构建

ACTION 表的构建规则：对每个 LALR(1)状态中的每个项目，若圆点不在右部末尾且圆点后为终结符 a ，则 $\text{ACTION}[\text{state}, a] = \text{shift}(\text{state_id})$ ；若圆点在右部末尾（归约项目），则对展望符 la 执行 $\text{ACTION}[\text{state}, la] = \text{reduce}(\text{prod_index})$ ；若归约项目为 $[S' \rightarrow S.]$ 且展望符为 $\$$ ，则 $\text{ACTION}[\text{state}, \$] = \text{accept}$ 。

GOTO 表：对每个状态，遍历其所有转移，若转移符号为非终结符 A ，则 $\text{GOTO}[\text{state}, A] = \text{target_state}$ 。

```

QMap<int, QMap<QString, ParsingAction>>
LRDFABuilder::buildActionTable(...) {
    for (const auto &state : lalrStates) {
        // 1) 移进：直接从转移表取终结符转移
        for (auto [sym, tgt] : state.transitions)
            if (terminals.contains(sym))
                actions[sym] = {"shift", tgt};
        // 2) 归约/接受：遍历归约项目
        for (const auto &item : state.items) {
            if (item.dotPos != prod.rhs.size()) continue;
            if (item.prodIndex == 0) // S' → S.
                actions["$"] = {"accept", -1};
            else
                actions[item.lookahead] = {"reduce", item.prodIndex};
        }
    }
}

```

```

    }
}
}

```

3.3.8 LALR(1) 分析器

LALR(1)分析器使用状态栈进行移进-归约分析。初始状态为0。每步读取当前状态和下一输入符号，查 ACTION 表得到动作：移进则推入目标状态并消费输入；归约则弹出 $|rhs|$ 个状态，查 GOTO 表推入新状态；接受则分析成功；出错则报告错误信息。

```

LRDFABuilder::ParseResult LRDFABuilder::parse(
    const QStringList &tokens,
    QMap<int, QMap<QString, ParsingAction>> &actionTable,
    QMap<int, QMap<QString, int>> &gotoTable,
    const QVector<Production> &augProductions)
{
    QVector<int> stateStack;
    stateStack.append(0);
    int pos = 0;
    while (true) {
        int state = stateStack.last();
        QString lookahead = (pos < tokens.size()) ? tokens[pos] : "$";
        const ParsingAction &act = actionTable[state][lookahead];
        if (act.type == "shift") {
            stateStack.append(act.value); pos++;
        } else if (act.type == "reduce") {
            int popCount = augProductions[act.value].rhs.size();
            stateStack.resize(stateStack.size() - popCount);

            stateStack.append(gotoTable[stateStack.last()][prod.lhs]);
        } else if (act.type == "accept")
            { accepted = true; break; }
    }
}

```

3.4 测试内容

3.4.1 测试文法一

$E \rightarrow E + T \mid T$
 $T \rightarrow a$

LALR(1) 语法分析生成器

文法输入

$E \rightarrow E + T \mid T$
 $T \rightarrow a$

开始分析
保存文法
打开文法
载入用例
清空

分析完成: 3 条产生式, 6 个 LR(0) 状态, 6 个 LR(1) 状态, 6 个 LALR(1) 状态

FIRST/FOLLOW 集

LR(0) DFA SLR(1) 检查 LR(1) DFA LALR(1) DFA LALR(1) 分析表 句子分析

FIRST 集

非终结符	FIRST 集
E	{ a }
T	{ a }

FOLLOW 集

非终结符	FOLLOW 集
E	{ \$, + }
T	{ \$, + }

FIRST/FOLLOW 集		LR(0) DFA	SLR(1) 检查	LR(1) DFA	LALR(1) DFA	LALR(1) 分析表	句子分析
状态	项目集	转移					
10 (初始)	$S' \rightarrow \cdot E$	$E \rightarrow I2; T \rightarrow I1; a \rightarrow I3$					
	$E \rightarrow \cdot E + T$	$E \rightarrow I2; T \rightarrow I1; a \rightarrow I3$					
	$E \rightarrow \cdot T$	$E \rightarrow I2; T \rightarrow I1; a \rightarrow I3$					
	$T \rightarrow \cdot a$	$E \rightarrow I2; T \rightarrow I1; a \rightarrow I3$					
11	$E \rightarrow T \cdot$						
12	$S' \rightarrow E \cdot$	$+ \rightarrow I4$					
	$E \rightarrow E \cdot + T$	$+ \rightarrow I4$					
13	$T \rightarrow a \cdot$						
14	$E \rightarrow E + \cdot T$	$T \rightarrow I5; a \rightarrow I3$					
	$T \rightarrow \cdot a$	$T \rightarrow I5; a \rightarrow I3$					

FIRST/FOLLOW 集	LR(0) DFA	SLR(1) 检查	LR(1) DFA	LALR(1) DFA	LALR(1) 分析表	句子分析
状态	移进符号	归约项目	FOLLOW 集 / 冲突说明			
结果：该文法是 SLR(1) 文法，无冲突。						
I0	{ a }					
I1		E -> T	{ \$, + }			
I2	{ + }					
I3		T -> a	{ \$, + }			
I4	{ a }					
I5		E -> E + T	{ \$, + }			

FIRST/FOLLOW 集	LR(0) DFA	SLR(1) 检查	LR(1) DFA	LALR(1) DFA	LALR(1) 分析表	句子分析
状态	项目集 (带展符号)	转移				
I0 (初始)						
	S' -> . E, \$	E -> I2; T -> I1; a -> I3				
	E -> . E + T, \$	E -> I2; T -> I1; a -> I3				
	E -> . T, \$	E -> I2; T -> I1; a -> I3				
	E -> . E + T, +	E -> I2; T -> I1; a -> I3				
	E -> . T, +	E -> I2; T -> I1; a -> I3				
	T -> . a, \$	E -> I2; T -> I1; a -> I3				
	T -> . a, +	E -> I2; T -> I1; a -> I3				
I1						
	E -> T ., \$					
	E -> T ., +					
I2						
	S' -> E ., \$	+ -> I4				
	E -> E . + T, \$	+ -> I4				
	F -> F . + T, +	+ -> I4				

FIRST/FOLLOW 集	LR(0) DFA	SLR(1) 检查	LR(1) DFA	LALR(1) DFA	LALR(1) 分析表	句子分析
状态	项目集 (合并展符号)	合并来源	转移			
I0		合并自 LR(1) 状态 0				
	S' -> . E, (\$)					
	E -> . E + T, (\$, +)					
	E -> . T, (\$, +)					
	T -> . a, (\$, +)		E -> I1; T -> I4; a -> I5			
I1		合并自 LR(1) 状态 2				
	S' -> E ., (\$)					
	E -> E . + T, (\$, +)		+ -> I2			
I2		合并自 LR(1) 状态 4				
	E -> E + . T, (\$, +)					
	T -> . a, (\$, +)		T -> I3; a -> I5			
I3		合并自 LR(1) 状态 5				
	E -> E + T ., (\$, +)					
I4		合并自 LR(1) 状态 1				
	E -> T ., (\$, +)					

FIRST/FOLLOW 集	LR(0) DFA	SLR(1) 检查	LR(1) DFA	LALR(1) DFA	LALR(1) 分析表	句子分析
状态	+	a	\$	E	T	
0	s5		1		4	
1	s2	acc				
2	s5				3	
3	r0	r0				
4	r1	r1				
5	r2	r2				

FIRST/FOLLOW 集LR(0) DFASLR(1) 检查LR(1) DFALALR(1) DFALALR(1) 分析表句子分析

输入待分析句子

a + a

句子符合文法，分析成功。

分析步骤

步骤	状态栈	输入	动作
0	0	a + a	初始化
1	0 5	+ a	移进到状态 5
2	0 4	+ a	归约: T -> a
3	0 1	+ a	归约: E -> T
4	0 1 2	a	移进到状态 2
5	0 1 2 5		移进到状态 5
6	0 1 2 3		归约: T -> a
7	0 1		归约: E -> E + T
8	0 1		接受

3.4.2 测试文法二

```
exp -> exp addop term | term
addop -> + | -
term -> term mulop factor | factor
mulop -> * | /
factor -> ( exp ) | n
```

FIRST/FOLLOW 集LR(0) DFASLR(1) 检查LR(1) DFALALR(1) DFALALR(1) 分析表句子分析

FIRST 集

非终结符	FIRST 集
addop	{ +, - }
exp	{ (, n }
factor	{ (, n }
mulop	{ *, / }
term	{ (, n }

FOLLOW 集

非终结符	FOLLOW 集
addop	{ (, n }
exp	{ \$,), +, -, / }
factor	{ \$,), +, -, / }
mulop	{ (, n }
term	{ \$,), +, -, / }

FIRST/FOLLOW 集	LR(0) DFA	SLR(1) 检查	LR(1) DFA	LALR(1) DFA	LALR(1) 分析表	句子分析
状态	项目集	转移				
I0 (初始)						
	S' -> . exp	(-> I2; exp -> I5; factor -> I3; n -> I1; term -> I4				
	exp -> . exp addop term	(-> I2; exp -> I5; factor -> I3; n -> I1; term -> I4				
	exp -> . term	(-> I2; exp -> I5; factor -> I3; n -> I1; term -> I4				
	term -> . term mulop factor	(-> I2; exp -> I5; factor -> I3; n -> I1; term -> I4				
	term -> . factor	(-> I2; exp -> I5; factor -> I3; n -> I1; term -> I4				
	factor -> . (exp)	(-> I2; exp -> I5; factor -> I3; n -> I1; term -> I4				
	factor -> . n	(-> I2; exp -> I5; factor -> I3; n -> I1; term -> I4				
I1						
	factor -> n .					
I2						
	factor -> (. exp)	(-> I2; exp -> I6; factor -> I3; n -> I1; term -> I4				
	exp -> . exp addop term	(-> I2; exp -> I6; factor -> I3; n -> I1; term -> I4				
	exp -> . term	(-> I2; exp -> I6; factor -> I3; n -> I1; term -> I4				
	term -> . term mulop factor	(-> I2; exp -> I6; factor -> I3; n -> I1; term -> I4				

FIRST/FOLLOW 集	LR(0) DFA	SLR(1) 检查	LR(1) DFA	LALR(1) DFA
状态	移进符号	归约项目		
结果：该文法是 SLR(1) 文法，无冲突。				

FIRST/FOLLOW 集	LR(0) DFA	SLR(1) 检查	LR(1) DFA	LALR(1) DFA	LALR(1) 分析表	句子分析
状态	项目集 (带lookahead)	转移				
I0 (初始)						
	S' -> . exp, \$	(-> I2; exp -> I5; factor -> I3; n -> I1; term -> I4				
	exp -> . exp addop term, \$	(-> I2; exp -> I5; factor -> I3; n -> I1; term -> I4				
	exp -> . term, \$	(-> I2; exp -> I5; factor -> I3; n -> I1; term -> I4				
	exp -> . exp addop term, -	(-> I2; exp -> I5; factor -> I3; n -> I1; term -> I4				
	exp -> . exp addop term, +	(-> I2; exp -> I5; factor -> I3; n -> I1; term -> I4				
	exp -> . term, -	(-> I2; exp -> I5; factor -> I3; n -> I1; term -> I4				
	exp -> . term, +	(-> I2; exp -> I5; factor -> I3; n -> I1; term -> I4				
	term -> . term mulop factor, \$	(-> I2; exp -> I5; factor -> I3; n -> I1; term -> I4				
	term -> . factor, \$	(-> I2; exp -> I5; factor -> I3; n -> I1; term -> I4				
	term -> . term mulop factor, -	(-> I2; exp -> I5; factor -> I3; n -> I1; term -> I4				
	term -> . factor, -	(-> I2; exp -> I5; factor -> I3; n -> I1; term -> I4				
	term -> . term mulop factor, +	(-> I2; exp -> I5; factor -> I3; n -> I1; term -> I4				
	term -> . factor, +	(-> I2; exp -> I5; factor -> I3; n -> I1; term -> I4				
	term -> . term mulop factor, /	(-> I2; exp -> I5; factor -> I3; n -> I1; term -> I4				

FIRST/FOLLOW 集	LR(0) DFA	SLR(1) 检查	LR(1) DFA	LALR(1) DFA	LALR(1) 分析表	句子分析
状态	项目集 (合并展望符)	合并来源	转移			
10	$S' \rightarrow \cdot \text{exp}, \{\$ \}$ $\text{factor} \rightarrow \cdot n, \{\$, *, +, -, /\}$ $\text{exp} \rightarrow \cdot \text{exp} \text{ addop term}, \{\$, +, -\}$ $\text{exp} \rightarrow \cdot \text{term}, \{\$, +, -\}$ $\text{term} \rightarrow \cdot \text{term mulop factor}, \{\$, *, +, -, /\}$ $\text{term} \rightarrow \cdot \text{factor}, \{\$, *, +, -, /\}$ $\text{factor} \rightarrow \cdot (\text{exp}), \{\$, *, +, -, /\}$	合并自 LR(1) 状态 0	$(\rightarrow 13; \text{exp} \rightarrow 11; \text{factor} \rightarrow 112; n \rightarrow 12; \text{term} \rightarrow 17$			
11	$S' \rightarrow \text{exp} \cdot \{\$ \}$ $\text{exp} \rightarrow \text{exp} \cdot \text{addop term}, \{\$, +, -\}$ $\text{addop} \rightarrow \cdot +, \{(\text{ n}\}$ $\text{addop} \rightarrow \cdot -, \{(\text{ n}\}$	合并自 LR(1) 状态 5	$+ \rightarrow 18; - \rightarrow 19; \text{addop} \rightarrow 15$			
12	$\text{factor} \rightarrow n \cdot \{\$,), *, +, -, /\}$	合并自 LR(1) 状态 1, 6				

FIRST/FOLLOW 集		LR(0) DFA		SLR(1) 检查		LR(1) DFA		LALR(1) DFA		LALR(1) 分析表		句子分析	
状态	(	)	*	+	-	/	n	\$	addop	exp	factor	mulop	term
0	s3						s2			1	12		7
1				s8	s9			acc	5				
2		r9	r9	r9	r9	r9	r9						
3	s3						s2			4	12		7
4		s15		s8	s9				5				
5	s3						s2				12		6
6		r0	s13	r0	r0	s14	r0					10	
7		r1	s13	r1	r1	s14	r1					10	
8	r2						r2						
9	r3						r3						
10	s3						s2				11		
11		r4	r4	r4	r4	r4	r4						
12		r5	r5	r5	r5	r5	r5						
13	r6						r6						
14	r7						r7						
**													

输入待分析句子

 开始分析

句子符合文法，分析成功。

分析步骤

步骤	状态栈	输入	动作
0	0	(n + n) / n	初始化
1	0 3	n + n) / n	移进到状态 3
2	0 3 2	+ n) / n	移进到状态 2
3	0 3 12	+ n) / n	归约: factor -> n
4	0 3 7	+ n) / n	归约: term -> factor
5	0 3 4	+ n) / n	归约: exp -> term
6	0 3 4 8	n) / n	移进到状态 8
7	0 3 4 5	n) / n	归约: addop -> +
8	0 3 4 5 2	) / n	移进到状态 2
9	0 3 4 5 12	) / n	归约: factor -> n

3.4.3 测试文法三

E -> E + T | T

T -> T * F | F

F -> (E) | id

FIRST/FOLLOW 集		LR(0) DFA	SLR(1) 检查	LR(1) DFA	LALR(1) DFA	LALR(1) 分析表	句子分析
FIRST 集							
非终结符	FIRST 集						
E	{ (, id }						
F	{ (, id }						
T	{ (, id }						
FOLLOW 集							
非终结符	FOLLOW 集						
E	{ \$,), + }						
F	{ \$,), *, + }						
T	{ \$,), *, + }						

FIRST/FOLLOW 集	LR(0) DFA	SLR(1) 检查	LR(1) DFA	LALR(1) DFA	LALR(1) 分析表	句子分析
状态	项目集	转移				
I0 (初始)						
	S' -> . E	(-> I5; E -> I4; F -> I2; T -> I3; id -> I1				
	E -> . E + T	(-> I5; E -> I4; F -> I2; T -> I3; id -> I1				
	E -> . T	(-> I5; E -> I4; F -> I2; T -> I3; id -> I1				
	T -> . T * F	(-> I5; E -> I4; F -> I2; T -> I3; id -> I1				
	T -> . F	(-> I5; E -> I4; F -> I2; T -> I3; id -> I1				
	F -> . (E)	(-> I5; E -> I4; F -> I2; T -> I3; id -> I1				
	F -> . id	(-> I5; E -> I4; F -> I2; T -> I3; id -> I1				
I1						
	F -> id .					
I2						
	T -> F .					
I3						
	E -> T .	* -> I6				
	T -> T . * F	* -> I6				
..						

FIRST/FOLLOW 集	LR(0) DFA	SLR(1) 检查	LR(1) DFA	LALR(1) DFA	LALR(1) 分析表	句子分析
状态		移进符号	归约项目	FOLLOW 集 / 冲突说明		
结果: 该文法是 SLR(1) 文法, 无冲突。						
I0	{ (, id }					
I1		F -> id	{ \$,), + }			
I2		T -> F	{ \$,), + }			
I3	*	E -> T	{ \$,), + }			
I4	{ + }					
I5	{ (, id }					
I6	{ (, id }					
I7	{ (, id }					
I8	{), + }					
I9		T -> T * F	{ \$,), + }			
I10	*	E -> E + T	{ \$,), + }			
I11		F -> (E)	{ \$,), + }			

FIRST/FOLLOW 集		LR(0) DFA	SLR(1) 检查	LR(1) DFA	LALR(1) DFA	LALR(1) 分析表	句子分析
状态	项目集 (带展缀符)	转移					
I19	$E \rightarrow E, T, +$	$\rightarrow I21; + \rightarrow I17$					
	$T \rightarrow T * F,)$						
	$T \rightarrow T * F, +$						
	$T \rightarrow T * F, *$						
I20	$E \rightarrow E + T,)$	$* \rightarrow I15$					
	$E \rightarrow E + T, +$	$* \rightarrow I15$					
	$T \rightarrow T, * F,)$	$* \rightarrow I15$					
	$T \rightarrow T, * F, +$	$* \rightarrow I15$					
	$T \rightarrow T, * F, *$	$* \rightarrow I15$					
I21	$F \rightarrow (E,)$						
	$F \rightarrow (E, +$						
	$F \rightarrow (E, *$						

FIRST/FOLLOW 集		LR(0) DFA	SLR(1) 检查	LR(1) DFA	LALR(1) DFA	LALR(1) 分析表	句子分析
状态	项目集 (合并展缀符)	合并来源	转移				
I0		合并自 LR(1) 状态 0					
	$S' \rightarrow, E, (\$, \$)$						
	$E \rightarrow, E + T, (\$, +)$						
	$E \rightarrow, T, (\$, +)$						
	$T \rightarrow, T * F, (\$, *, +)$						
	$T \rightarrow, F, (\$, *, +)$						
	$F \rightarrow, (E), (\$, *, +)$						
	$F \rightarrow, id, (\$, *, +)$		$(\rightarrow I2; E \rightarrow I1; F \rightarrow I9; T \rightarrow I6; id \rightarrow I11$				
I1	$S' \rightarrow E,) (\$, \$)$	合并自 LR(1) 状态 4					
	$E \rightarrow E, + T, (\$, +)$		$+ \rightarrow I4$				
I2		合并自 LR(1) 状态 5, 12					
	$E \rightarrow, E + T, (), +)$						
	$E \rightarrow, T, (), +)$						
	$T \rightarrow, T * F, (), *, +)$						

FIRST/FOLLOW 集		LR(0) DFA		SLR(1) 检查		LR(1) DFA		LALR(1) DFA		LALR(1) 分析表		句子分析	
状态	(	)	*	+	id	\$	E	F	T				
0	s2				s11		1	9	6				
1				s4		acc							
2	s2				s11		3	9	6				
3		s10		s4									
4	s2				s11			9	5				
5		r0	s7	r0		r0							
6		r1	s7	r1		r1							
7	s2				s11			8					
8		r2	r2	r2		r2							
9		r3	r3	r3		r3							
10		r4	r4	r4		r4							
11		r5	r5	r5		r5							

FIRST/FOLLOW 集LR(0) DFASLR(1) 检查LR(1) DFALALR(1) DFALALR(1) 分析表句子分析

输入待分析句子

id * (id + id)

开始分析

句子符合文法，分析成功。

分析步骤

步骤	状态栈	输入	动作
0	0	id * (id + id)	初始化
1	0 11	* (id + id)	移进到状态 11
2	0 9	* (id + id)	归约: F -> id
3	0 6	* (id + id)	归约: T -> F
4	0 6 7	(id + id)	移进到状态 7
5	0 6 7 2	id + id)	移进到状态 2
6	0 6 7 2 11	+ id)	移进到状态 11
7	0 6 7 2 9	+ id)	归约: F -> id
8	0 6 7 2 6	+ id)	归约: T -> F
9	0 6 7 2 3	+ id)	归约: E -> T

3.4.4 测试文法四

S -> i E t S | i E t S e S | a
E -> b

分析完成: 4 条产生式, 10 个 LR(0) 状态, 17 个 LR(1) 状态, 10 个 LALR(1) 状态

FIRST/FOLLOW 集LR(0) DFASLR(1) 检查LR(1) DFALALR(1) DFALALR(1) 分析表句子分析

FIRST 集

非终结符	FIRST 集
E	{ b }
S	{ a, i }

FOLLOW 集

非终结符	FOLLOW 集
E	{ t }
S	{ \$, e }

FIRST/FOLLOW 集		LR(0) DFA	SLR(1) 检查	LR(1) DFA	LALR(1) DFA	LALR(1) 分析表	句子分析
状态	项目集	转移					
I0 (初始)							
	S' -> . S	S -> I2; a -> I3; i -> I1					
	S -> . i E t S	S -> I2; a -> I3; i -> I1					
	S -> . i E t S e S	S -> I2; a -> I3; i -> I1					
	S -> . a	S -> I2; a -> I3; i -> I1					
I1							
	S -> i . E t S	E -> I4; b -> I5					
	S -> i . E t S e S	E -> I4; b -> I5					
	E -> . b	E -> I4; b -> I5					
I2							
	S' -> S .						
I3							
	S -> a .						
I4							
	S -> i E . t S	t -> I6					

FIRST/FOLLOW 集		LR(0) DFA	SLR(1) 检查	LR(1) DFA	LALR(1) DFA	LALR(1) 分析表	句子分析
状态	移进符号	归约项目	FOLLOW 集 / 冲突说明				
结果: 该文法不是 SLR(1) 文法。							
冲突			状态 7: 在 'e' 上 shift 与 reduce 冲突 (S -> i E t S)				
..	...						

FIRST/FOLLOW 集		LR(0) DFA	SLR(1) 检查	LR(1) DFA	LALR(1) DFA	LALR(1) 分析表	句子分析
状态	项目集 (带展缩符)	转移					
	S -> i E t S . e S, \$	e -> I15					
	S -> i E t S . e	e -> I15					
	S -> i E t S . e S, e	e -> I15					
I15							
	S -> i E t S e . S, \$	S -> I16; a -> I9; i -> I7					
	S -> i E t S e . S, e	S -> I16; a -> I9; i -> I7					
	S -> . i E t S, \$	S -> I16; a -> I9; i -> I7					
	S -> . i E t S e S, \$	S -> I16; a -> I9; i -> I7					
	S -> . a, \$	S -> I16; a -> I9; i -> I7					
	S -> . i E t S, e	S -> I16; a -> I9; i -> I7					
	S -> . i E t S e S, e	S -> I16; a -> I9; i -> I7					
	S -> . a, e	S -> I16; a -> I9; i -> I7					
I16							
	S -> i E t S e S ., \$						
	S -> i E t S e S ., e						

FIRST/FOLLOW 集	LR(0) DFA	SLR(1) 检查	LR(1) DFA	LALR(1) DFA	LALR(1) 分析表	句子分析
状态	项目集 (合并展望符)	合并来源	转移			
10		合并自 LR(1) 状态 0				
	S' -> . S, (\$)					
	S -> . i E t S, (\$)					
	S -> . i E t S e S, (\$)					
	S -> . a, (\$)		S -> 11; a -> 18; i -> 14			
11		合并自 LR(1) 状态 2				
	S' -> S ., (\$)					
12		合并自 LR(1) 状态 6, 12				
	S -> . i E t S, (\$, e)					
	S -> i E t . S, (\$, e)					
	S -> . i E t S e S, (\$, e)					
	S -> i E t . S e S, (\$, e)					
	S -> . a, (\$, e)		S -> 16; a -> 18; i -> 14			
13		合并自 LR(1) 状态 11, 15				
	S -> . i E t S, (\$, e)					

FIRST/FOLLOW 集		LR(0) DFA		SLR(1) 检查		LR(1) DFA		LALR(1) DFA		LALR(1) 分析表		句子分析	
状态	a	b	e	i	t	\$	E	S					
0	s8		s4					1					
1					acc								
2	s8		s4					6					
3	s8		s4					7					
4		s9					5						
5					s2								
6			冲突			r0							
7			r1			r1							
8			r2			r2							
9				r3									

四、实验总结（心得体会）

通过本次实验，我深入理解和实现了 LR 语法分析的全套算法流程，从最初的文法解析、FIRST/FOLLOW 集计算，到 LR(0) DFA 构建、SLR(1) 冲突检测，再到 LR(1) DFA 构建和 LALR(1)同核状态合并，最后到分析表生成和句子分析，完整地走通了 LALR(1)语法分析生成器的整个技术路线。

在实现过程中，最需要关注的是 LR(1)项目展望符的传播机制。在闭包操作中，展望符的计算需要用到 FIRST(β a)，这要求正确理解前瞻符号在 LR(1)分析中的核心作用。另一个技术难点是 LALR(1)的同核状态合并，合并后必须仔细检测是否产生新的冲突，这决定了文法是否可被 LALR(1)方法处理。

此外，通过对比四种文法的状态数量差异，能够直观地看到 LR(1)比 LR(0)具有更强的分析能力（通过展望符减少冲突），而 LALR(1)通过合并状态在保持与 LR(1)相同分析能力的同时大大减少了状态数量，是一种兼顾效率和能力的实用文法。

本次实验使用 C++和 Qt 6 框架开发，将编译原理理论与实践相结合，不仅加深了对 LR 语法分析的理解，也提高了实际软件工程开发能力。

五、参考文献：

[【编译原理】一篇搞定 LR 分析法 \(LR\(1\)、LR\(0\)、SLR、LALR\)-CSDN 博客](#)

[3 编译原理如何求 first 集和 follow 集（更正版）_哔哩哔哩_bilibili](#)

[7 编译原理构造 LR\(0\)分析表\(包含构建项目规范族\)_哔哩哔哩_bilibili](#)

[9 编译原理构造 LR\(1\)分析表（带向前搜索符的项目集规范族）_哔哩哔哩_bilibili](#)

[11 编译原理如何区别 LR\(0\),SLR\(1\),LR\(1\),LALR\(1\)文法_哔哩哔哩_bilibili](#)